Imagina

Clean Architecture para Frontend con React y Node

Joan Cifo

Sesión 2 · Miércoles 11 de noviembre de 2026 · 17:00 – 20:00

Índice de la sesión

5 MIN

Del diagnóstico de ayer a reglas que impiden que la deuda vuelva a entrar

Hora	Bloque	Contenido
17:00	**Apertura**	Repaso de la Sesión 1, ranking y repositorio imagina-clean-react-s2
17:15	**Tema 3**	Acoplamiento, cohesión y principios SOLID aplicados a React y TypeScript
18:00	**Tema 4**	Capas, responsabilidades, frameworks y drivers en Clean Architecture
18:45	**Pausa**	15 minutos
19:00	**Ejercicio**	Proteger el núcleo: reglas de importación y un caso de uso con su doble de prueba
19:30	**Corrección**	Solución comentada y errores frecuentes
19:45	**Cierre**	Proyecto final · Test de conceptos · Feedback individual

Repaso relámpago y ranking

5 MIN

Cinco preguntas sobre la Sesión 1 · Responde en el chat todo a la vez: 1B, 2A…

DINÁMICA · 1 PUNTO POR ACIERTO

1. En Clean Architecture, las dependencias de código apuntan…

A) hacia fuera B) hacia dentro C) hacia la interfaz

2. ¿En cuántos sitios calcula OrderDesk el total de un pedido?

A) 1 B) 2 C) 3

3. Un valor calculable a partir de props o estado…

A) se sincroniza con useEffect B) se calcula durante el render

4. ¿Por qué pasaban los 9 tests tras borrar los permisos?

A) comprobaban CSS y snapshot B) los permisos estaban en el backend

5. Prioridad en la matriz de deuda =

A) (Impacto + Riesgo) × Facilidad B) Impacto × Riesgo

Ranking

Acumulado de dinámicas y tests. Se anuncia al inicio de cada sesión.

Puntuable hoy

- Repaso (5)
- Tipos de dependencia (5)
- ¿Abstracción o ceremonia? (4)
- ¿En qué capa va? (8)
- Test de conceptos (5)

CLAVE Respuestas: 1B · 2C · 3B · 4A · 5A. Si fallaste la 3, revisa react.dev «You Might Not Need an Effect».

Repositorio de la Sesión 2

5 MIN

OrderDesk con el material de hoy: ejemplos por principio y una feature por capas

PUESTA EN MARCHA

```
# Terminal de Visual Studio Code (Ctrl+ñ)
$ git clone \
    https://github.com/joancifo/imagina-clean-react-s2.git
$ cd imagina-clean-react-s2
$ code .
$ npm install
$ npm test
  Test Files  14 passed | 1 skipped (15)
       Tests  36 passed | 8 skipped (44)

# Ejemplos de hoy
$ npm run demo:tema3
$ npm run demo:tema4
```

Novedad	Para qué
`examples/tema3/`	Un ejemplo por principio SOLID
`src/orders/`	Feature Pedidos por capas
`eslint.architecture.js`	Reglas de importación
`docs/checklist-pr.md`	Checklist SOLID para revisar
`.github/instructions/`	Instrucciones de Copilot por ruta

MIGRACIÓN El código heredado no cambia: src/orders convive con él. Así se migra en un proyecto real, sin reescribir.

03

Acoplamiento, cohesión y principios SOLID aplicados a React y TypeScript

Tema 3 · 17:15 – 18:00

Coste de cambio · Una razón para existir · SOLID con ejemplos ejecutables

Imagina

Acoplamiento: el coste de cambiar una pieza

5 MIN

Cuántos módulos se rompen o hay que revisar cuando cambia uno

EXPERIMENTO · EL BACKEND RENOMBRA ord_st

```
# ¿Quién conoce los nombres del backend?
$ grep -rlE "ord_st|cust_nm|ln_items|crt_ts|owner_id" \
    src --include=*.ts --include=*.tsx | grep -v test
src/components/OrderRow.tsx
src/pages/DashboardPage.tsx
src/pages/OrderDetailPage.tsx
src/pages/OrdersPage.tsx
src/shared/mappers.ts
src/shared/types.ts
src/shared/utils.ts
src/orders/infrastructure/order-dto.ts
src/orders/infrastructure/order-mapper.ts

# Heredado: 7 archivos · src/orders: 2, ambos en infrastructure
```

1 Salientes (eferente)

¿De cuántos módulos depende? Muchas → el archivo se rompe por cambios ajenos.

2 Entrantes (aferente)

¿Cuántos dependen de él? Muchas → cambiarlo es caro. types.ts: 8.

Medirlo

`npm run map`

Columnas «salientes» y «entrantes».

IDEA Un límite bien puesto concentra el cambio: si el backend renombra un campo, cambia el mapper y nada más.

Aceptable, accidental o peligrosa

5 MIN

No todas las dependencias pesan igual: la pregunta es si van en la dirección correcta

A Aceptable

Estable y en la dirección correcta.

Componente → React
Dominio → dominio
Hook → caso de uso

B Accidental

Por comodidad, no por diseño. Propaga cambios.

utils.ts → api.ts (log)
Fila → Order del backend

C Peligrosa

Rompe la regla de dependencia o crea ciclos.

Regla → localStorage
Caso de uso → axios
api.ts ↔ utils.ts

DINÁMICA · CLASIFICA A, B O C

1. `OrderRow.tsx → ./Badge`
2. `utils.ts → services/api`
3. `OrderRow.tsx → shared/types (Order)`
4. `OrdersPage.tsx → fetch(…/approve)`
5. `useOrders.ts → store/appStore`

CLAVE Respuestas: 1A · 2C · 3B · 4C · 5B. Se valora el motivo: dirección de la dependencia y qué cambio propaga.

Cohesión: una sola razón para existir

5 MIN

Todo lo que contiene un módulo debería cambiar por el mismo motivo

Función en shared/utils.ts	¿Quién pide el cambio?	Ejemplo de cambio	Destino con cohesión
`formatCurrency`	Presentación (UX)	Mostrar «EUR» en vez de «€»	ui/formatters
`formatDate`	Presentación (UX)	Fecha relativa: «hace 2 días»	ui/formatters
`calcOrderTotal`	**Negocio (finanzas)**	IVA reducido para un producto	**domain/order.ts (ya existe)**
`getErrorMessage`	Infraestructura	Leer errores con otro formato	infrastructure (errores)
`trackEvent`	Analítica (marketing)	Cambiar de proveedor	Puerto + adapter (Sesión 4)

4 motivos de cambio · 7 módulos lo importan → cada cambio obliga a revisar 7 archivos.

Common Closure Principle: agrupa lo que cambia a la vez y por el mismo motivo.

REFLEXIÓN ¿Cuántos motivos de cambio conviven en la carpeta utils o shared de tu proyecto real?

S ·Single Responsibility en un handler

5 MIN

«A module should be responsible to one, and only one, actor.» — R. C. Martin, Clean Architecture (2017)

ANTES · UNA FUNCIÓN, CUATRO ACTORES

```
// OrdersPage.tsx:75-111 (resumido)
const handleApprove = async (order) => {
  if (!user) alert(…);                  // 1 sesión
  if (role !== 'admin' && …) alert(…); // 2 permisos
  if (order.ord_st !== 'pending') …;    // 3 validación
  if (total > 10000 && …) alert(…);     // 4 regla
  const res = await fetch(…);           // 5 HTTP
  trackEvent('order_approved', …);      // 6 analítica
  if (total > 5000) navigate(…);        // 7 navegación
};
// 4 actores: negocio · backend · analítica · UX
```

DESPUÉS · SOLO LA DECISIÓN DE NEGOCIO

```
// examples/tema3/srp-approval.ts
export function decideApproval(i: ApprovalInput)
  : ApprovalDecision {
  if (i.role === null) return no('no-session');
  if (i.role === 'agent') return no('forbidden');
  if (i.status !== 'pending') return no('not-pending');
  if (!validLines(i.lines)) return no('invalid-lines');
  if (i.total > 10000 && i.role !== 'admin')
    return no('needs-admin');
  return { ok: true };
}
// Sin alert, sin red, sin navegación
$ npm run demo:tema3   ✓ srp-approval (4 tests)
```

MATIZ Las líneas «no(…)» abrevian { ok: false, reason }. SRP no es «archivos pequeños»: es separar lo que cambia por motivos distintos.

O ·Open/Closed: añadir sin modificar

5 MIN

Abierto a extensión, cerrado a modificación — B. Meyer (1988), reformulado por R. C. Martin

ANTES · HAY QUE MODIFICAR PARA EXTENDER

```
// src/components/Badge.tsx:2
const label = status === 'pending' ? 'Pendiente'
  : status === 'approved' ? 'Aprobado'
  : status === 'shipped' ? 'Enviado'
  : 'Cancelado'; // on_hold → «Cancelado»😶
```

DESPUÉS · EXTENDER = AÑADIR UNA ENTRADA

```
// examples/tema3/ocp-order-status.ts
export const STATUS_VIEW = {
  pending:   { label: 'Pendiente', tone: 'neutral' },
  approved:  { label: 'Aprobado',  tone: 'neutral' },
  shipped:   { label: 'Enviado',   tone: 'neutral' },
  cancelled: { label: 'Cancelado', tone: 'danger' },
} satisfies Record<OrderStatus, StatusView>;
```

EXPERIMENTO · EL COMPILADOR AVISA

```
# Añade | 'on_hold' a OrderStatus
$ npm run typecheck
error TS1360: … does not satisfy the expected
type 'Record<OrderStatus, StatusView>'.
Property 'on_hold' is missing …
```

En React: composición

children o un hueco de acciones en lugar de booleanos (isAdmin, isCompact…): se extiende sin tocar el componente.

DOCS satisfies (TypeScript 4.9) valida contra un tipo sin perder el tipo literal inferido. Úsalo para tablas por estado, rol o nivel.

L ·Liskov: el contrato es también comportamiento

5 MIN

Una implementación debe poder sustituir a otra sin que el consumidor lo note — B. Liskov (1987)

MISMA FIRMA · DISTINTA PROMESA

```
/** Devuelve el pedido o null. No lanza. */
interface OrderReader {
  findById(id: string): Promise<Order | null>;
}

// Cumple
findById: async (id) =>
  orders.find((o) => o.id === id) ?? null

// Misma firma, otro comportamiento
findById: async (id) => {
  const o = orders.find((x) => x.id === id);
  if (!o) throw new Error('no encontrado');
  return o;
}
```

TEST DE CONTRATO · examples/tema3/lsp-...

```
function orderReaderContract(name, create) {
  it('devuelve null si no existe', async () =>
    expect(create(data).findById('P-9'))
      .resolves.toBeNull());
}
orderReaderContract('memoria', createInMemory…);
it.fails('estricta: devuelve null…', …);
```

También en componentes

<PrimaryButton> que ignora disabled no sustituye a <button>: el formulario se envía dos veces.

CLAVE TypeScript comprueba la forma; el comportamiento lo comprueba un test de contrato contra cada implementación.

I · Interface Segregation: pedir solo lo que se usa

5 MIN

Ningún cliente debería depender de métodos o datos que no utiliza

COMPONENTE · RECIBE DE TODO

```
// src/components/OrderRow.tsx:5-21
type Props = { order: Order; // DTO completo
  showTotal; showDate; showOwner; isAdmin;
  isManager; isCompact; isHighlighted;
  canApprove; canCancel; disabled; … } // 15 props
```

CONTEXTO · EXPONE DE TODO

```
// src/store/appStore.tsx:4-15 → 10 miembros
const { user, filters, setFilters,
        toast, setToast } = useAppStore();
// Cambia toast → re-render de todos los
// componentes que leen el contexto
```

DESPUÉS · VIEW MODEL DE 4 CAMPOS

```
// examples/tema3/isp-order-row.tsx
type OrderRowView = {
  id: string; customer: string;
  statusLabel: string; total: string; // formateado
};
export function OrderRowLite(
  { row, actions }: { row: OrderRowView;
                      actions?: ReactNode })
```

Herramientas

`Pick<Order, 'id' | 'status'>`

View model por pantalla · contextos separados por motivo de cambio

DOCS react.dev · useContext: React vuelve a renderizar los componentes que leen un contexto cuando su valor cambia.

D ·Dependency Inversion: el núcleo define el puerto

5 MIN

Los detalles dependen de las políticas, no al revés



las dos flechas apuntan al puerto

PUERTO · CASO DE USO · ADAPTER (SIMPLIFICADO)

```
// examples/tema3/dip-load-orders.ts
export interface OrderReader { findAll(): Promise<Order[]> }   // puerto

export async function countPendingOrders(orders: OrderReader) {
  return (await orders.findAll()).filter((o) => o.status === 'pending').length;
}
export function createHttpOrderReader(fetchFn: typeof fetch): OrderReader {
  return { findAll: async () => (await (await fetchFn('/api/orders')).json())
    .map(toOrder) };                      // DTO del backend → modelo interno
}
```

PRUEBA SIN RED

```
// test con un fake
const fake: OrderReader = {
  findAll: async () => [
    { id: 'P-1',
      status: 'pending',
      lines: [] },
  ],
};
expect(await
  countPendingOrders(fake))
  .toBe(1);
// sin red · sin React
```

CLAVE El puerto lo define y lo posee quien lo usa (el núcleo). El adapter lo implementa y recibe sus propios detalles (fetch).

¿Abstracción o ceremonia?

5 MIN

Cada interfaz tiene un coste: un archivo más, una indirección más y algo más que mantener

DINÁMICA · S (APORTA) O N (CEREMONIA)

1. IOrderFormatter con una sola implementación, usada en un componente

2. Puerto OrderReader entre caso de uso y fetch

3. Hook useStateManager que envuelve useState

4. Adapter de analítica (hoy console, mañana un proveedor)

Aporta si… cruza un límite de capa, tiene 2+ implementaciones (contando tests) o aísla algo volátil.

Checklist SOLID del Pull Request	
S	¿Cuántos actores pueden pedir cambios en este archivo?
O	Un estado/rol nuevo, ¿obliga a editar if/else dispersos?
L	¿Cada implementación supera el test de contrato?
I	¿Recibe datos o props que no usa?
D	¿El núcleo importa fetch, axios, React o localStorage?
+	¿Interfaz nueva con una implementación y sin límite?

CLAVE Respuestas: 1N · 2S · 3N · 4S. Checklist completa: docs/checklist-pr.md y .github/pull_request_template.md.

04

Capas, responsabilidades, frameworks y drivers en Clean Architecture

Tema 4 · 18:00 – 18:45

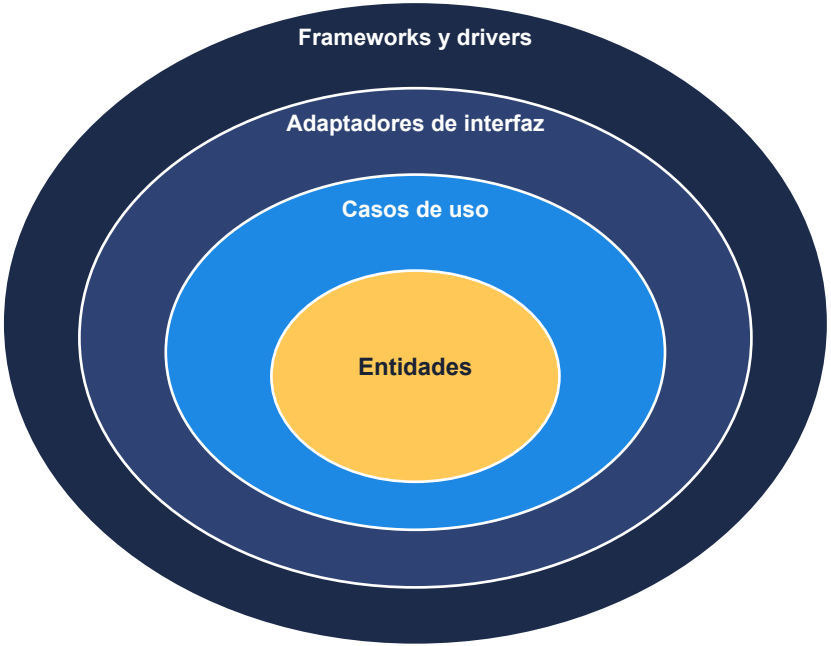
Cuatro anillos · Qué va en cada capa · Drivers · Reglas que se comprueban solas

Imagina

Los cuatro anillos aplicados a frontend

5 MIN

Terminología canónica de R. C. Martin (2012) y su traducción a src/orders



Anillo	En src/orders
Entidades	domain/order.ts · Order, calculateTotal, isUrgent
Casos de uso	application/ · loadOrderSummary, ports.ts
Adaptadores	infrastructure/ (HTTP, DTO, mapper) · ui/ (hooks, view models)
Frameworks y drivers	React, React Router, fetch, Vite, Express
Main	composition.ts · conecta implementaciones

REGLA Hacia dentro se sabe cada vez menos: el dominio no sabe que existe React; React no debería contener reglas de negocio.

Qué va en cada capa de src/orders

5 MIN

Una estructura pequeña, con contratos explícitos y sin capas vacías

ESTRUCTURA (SIN TESTS)

```
src/orders/
├─ domain/
│  └─ order.ts
├─ application/
│  ├─ ports.ts
│  ├─ load-order-summary.ts
│  └─ list-urgent-orders.ts  ← ejercicio
├─ infrastructure/
│  ├─ order-dto.ts
│  ├─ order-mapper.ts
│  ├─ http-order-reader.ts
│  └─ in-memory-order-reader.ts
├─ ui/
│  └─ useOrderSummary.ts
└─ composition.ts
```

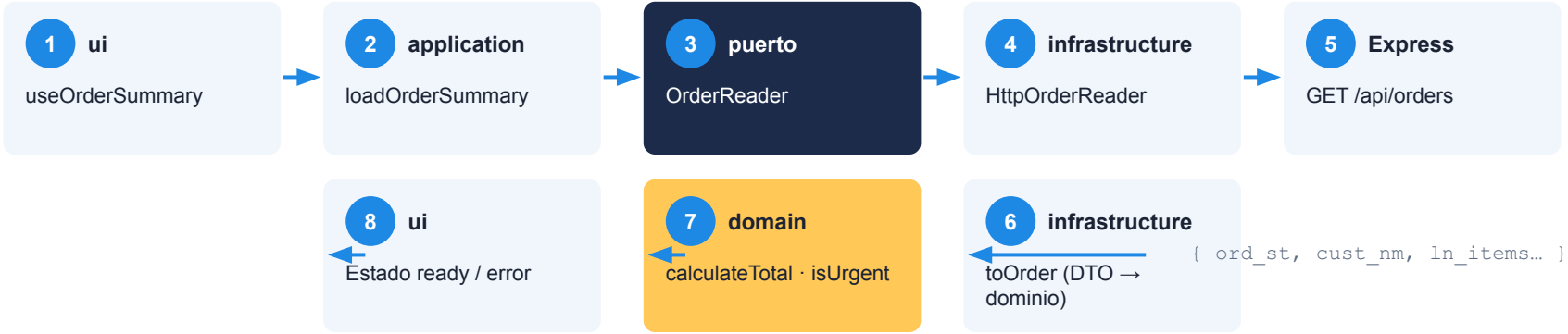
Capa	Contiene	Puede importar
`domain`	Entidades, value objects, reglas, tipos	—
`application`	Casos de uso y puertos	domain
`infrastructure`	HTTP, storage, analytics, SDKs, DTO y mappers	domain, application
`ui`	Componentes, view models, hooks adaptadores	domain, application, composition
`composition`	Elige y conecta implementaciones	todo

CRITERIO Sin carpetas vacías «por si acaso»: una capa existe cuando hay algo que proteger. Mañana: ¿capas globales o por feature?

Recorrido de una petición por las capas

5 MIN

Del render de React a la API y de vuelta, en src/orders



PRESENTACIÓN · HOOK ADAPTADOR

```
// src/orders/ui/useOrderSummary.ts
export type OrderSummaryState = { status: 'loading' } | { status: 'ready'; summary: OrderSummary } | { status: 'error' };
export function useOrderSummary(reader: OrderReader = orderReader): OrderSummaryState {   // reader ← composition.ts
  useEffect(() => { loadOrderSummary(reader).then(…ready…).catch(…error…) }, [reader]); …
```

BORDE El mapper valida en tiempo de ejecución: un estado desconocido falla en el borde (paso 6), no tres pantallas después.

Entrada, salida y dónde vive cada tecnología

5 MIN

Adaptadores primarios (conducen la aplicación) y secundarios (la aplicación los conduce)

Entrada · primarios

- Componentes y handlers de React
- Router (ruta → pantalla)
- Tests que llaman a casos de uso
- Rutas Express de un BFF
- Scripts y CLI de Node
- Llaman al núcleo directamente

Salida · secundarios

- HTTP (fetch, axios)
- localStorage / sessionStorage
- Analítica y logging
- Feature flags
- Reloj, generador de ids
- Detrás de un puerto del núcleo

Tecnología	Ubicación
React, Router	Framework · usado en ui
fetch, axios	infrastructure
localStorage	infrastructure (puerto)
Base de datos	Infra del BFF, no del front
SDKs externos	infrastructure
Sistema de archivos	Scripts Node de tooling
CI (Actions)	Fuera: ejecuta controles

ORIGEN Primario/secundario y driving/driven proceden de Ports & Adapters (A. Cockburn, 2005). Clean Architecture integra la idea.

Frameworks como detalles, con coste real

5 MIN

El núcleo se prueba sin React, sin navegador y sin red

DEMO · EL NÚCLEO SIN REACT

```
// @vitest-environment node
// examples/tema4/core-without-react.test.ts
it('no hay navegador en este entorno', () => {
  expect(typeof window).toBe('undefined');
  expect(typeof localStorage).toBe('undefined');
});
it('el caso de uso calcula el resumen', async () => {
  const summary = await loadOrderSummary(
    createInMemoryOrderReader(orders));
  expect(summary.pendingCount).toBe(1);
});

$ npm run demo:tema4
 ✓ examples/tema4/core-without-react.test.ts (2)
 ✓ src/orders/application/load-order-summary…
```

1 Reglas en milisegundos

Dominio y casos de uso, sin montar componentes.

2 Cambiar axios por fetch

Solo cambia infrastructure.

3 El backend renombra campos

Solo cambian el DTO y el mapper.

Sustituible ≠ gratis

Cambiar de framework sigue costando ui y tests de componentes.

HONESTIDAD No se diseña para migrar de framework cada año, sino para que lo que cambia a menudo no arrastre al núcleo.

La regla de dependencia, comprobada por ESLint

5 MIN

no-restricted-imports y no-restricted-globals por capa en eslint.architecture.js

UN BLOQUE POR CAPA

```
// eslint.architecture.js (bloque del dominio)
{
  files: ['src/orders/domain/**/*.{ts,tsx}'],
  rules: {
    'no-restricted-imports': ['error', { patterns: [
      FRAMEWORKS,              // react, react-router-dom, axios…
      layer('application'),    // **/application, **/application/**
      layer('infrastructure'), layer('ui'), composition] }],
    'no-restricted-globals': ['error', ...BROWSER_GLOBALS],
  },                // window, document, localStorage, fetch, alert
}
```

PRUEBA DE FUEGO

```
$ npm run lint:arch
src/orders/domain/order.ts  1:1  error  'axios' import is restricted
… El núcleo (dominio y aplicación) no depende de frameworks…
```

1 Patrones = texto del import

`**/infrastructure/**`

coincide con ../infrastructure/x

2 Mensajes propios

El error explica la regla a quien llegue dentro de seis meses.

3 Alcance: src/orders

El legado no se bloquea; la regla se amplía al migrar.

DOCS eslint.org: no-restricted-imports y no-restricted-globals. Más potencia: eslint-plugin-boundaries o dependency-cruiser.

Las reglas también se prueban

5 MIN

Una regla que nadie ha visto fallar no protege nada

VIOLACIÓN DETECTADA · SIN FALSOS POSITIVOS

```
// src/orders/architecture.test.ts
const eslint = new ESLint({
  overrideConfigFile: 'eslint.architecture.js' });

async function errorsFor(filePath, code) {
  const [r] = await eslint.lintText(code, { filePath });
  return r.messages.map((m) => m.ruleId);
}

it('dominio: no puede importar React', async () =>
  expect(await errorsFor('src/orders/domain/x.ts',
    "import { useState } from 'react';"))
    .toContain('no-restricted-imports'));

it('aplicación: sí puede importar el dominio', …toEqual([]));
```

HOY EN TU REPOSITORIO

```
$ npm run test:arch
 ✓ architecture.test.ts
   (9 tests | 5 skipped)
→ los 5 son tuyos a las 19:00
```

Orden de trabajo

- 1. Test que describe la regla
- 2. Regla en ESLint
- 3. Prueba de fuego con una violación
- 4. CI la ejecuta (Sesión 9)

PREGUNTA ¿Qué pasaría si alguien borra el bloque del dominio y no existiera este test?

¿En qué capa va?

5 MIN

Ocho responsabilidades reales de OrderDesk · 1 punto por acierto

DINÁMICA · D · A · I · P · C

1. **«Más de 10000 € lo aprueba un administrador»**
2. **«Aprobar pedido»: leer, aplicar la regla y guardar**
3. **Traducir ord_st → status**
4. **Hook useOrderSummary**
5. **Elegir el lector HTTP o el de memoria**
6. **Guardar los filtros en localStorage**
7. **Formatear 1234.5 como «1.234,50 €»**
8. **Convertir un HTTP 500 en OrderReadError**

CLAVE Respuestas: 1D · 2A · 3I · 4P · 5C · 6I · 7P · 8I. Que los filtros se guarden es producto; dónde se guardan, infraestructura.

Copilot al servicio de las capas

5 MIN

Instrucciones por ruta que guían; reglas y tests que garantizan

INSTRUCCIONES POR RUTA

```
# .github/instructions/orders-core.instructions.md
---
applyTo: "src/orders/domain/**,src/orders/application/**"
---
# Núcleo de pedidos (dominio y aplicación)
- TypeScript puro: sin React, router, axios,
  fetch, window ni localStorage.
- Casos de uso con entrada y salida explícitas
  y dependencias tipadas con puertos.
- Cada caso de uso nuevo llega con tests en
  entorno node y un doble del puerto.
```

PROMPT P1 · prompts/sesion-2.md

```
#file:src/components/OrderRow.tsx
Revisa este componente con SOLID.
Para cada principio: ¿se cumple, se
incumple o no aplica? Evidencia
ruta:línea, consecuencia sobre el coste
de cambio y la alternativa MÁS PEQUEÑA
posible. No propongas interfaces nuevas
salvo que crucen un límite de capa.

# Chat: Ctrl+Alt+I · ^⌘I · modo Ask
```

Revisa cada respuesta

¿Existe la evidencia ruta:línea? ¿Añade imports que lint:arch rechazaría? ¿Crea una interfaz con una sola implementación?

LÍMITE Las instrucciones guían, no garantizan. La garantía es npm run lint:arch && npm run test:arch antes de cada Pull Request.

15

Pausa ·15 minutos

Volvemos a las 19:00 con el ejercicio
Para pensar: ¿qué import de tu proyecto real rechazaría hoy eslint.architecture.js?

Parte A: reglas de importación por capa

15 MIN

Individual · 19:00 – 19:15 · Enunciado completo en docs/ejercicio-s2.md

1 · PUNTO DE PARTIDA

```
$ git checkout -b s2/<tu-nombre>
$ npm run test:arch      # 4✓ · 5 omitidos

// eslint.architecture.js
// APLICACIÓN: solo dominio y sus puertos.
// TODO (Parte A): files 'src/orders/application/…'
//   no-restricted-imports → FRAMEWORKS,
//     layer('infrastructure'), layer('ui'),
//     composition
//   no-restricted-globals → ...BROWSER_GLOBALS
// INFRAESTRUCTURA: TODO → layer('ui'), composition
// PRESENTACIÓN:    TODO → layer('infrastructure'),
//                         axios
```

2 Escribe las tres reglas

Copia el bloque del dominio y adapta files y patrones. La aplicación sí importa domain.

3 Activa los tests

Quita .skip de «aplicación» y «presentación» → **`npm run test:arch`**

4 Prueba de fuego

`import axios from 'axios';`

en application/ports.ts → npm run lint:arch → lee el error → deshaz

CRITERIO Terminado = 9 tests de test:arch en verde y la prueba de fuego vista. Si acabas antes, pasa a la Parte B.

Parte B: un caso de uso con su doble de prueba

15 MIN

Individual · 19:15 – 19:30 · src/orders/application/list-urgent-orders.ts

PUNTO DE PARTIDA

```
// list-urgent-orders.test.ts · paso 1
function createFakeOrderReader(orders: Order[])
  : OrderReader {
  throw new Error('Por implementar');
}
describe.skip('listUrgentOrders', …) // paso 2

// list-urgent-orders.ts · paso 3
export type UrgentOrder =
  { id: string; customerName: string; total: number };
export async function listUrgentOrders(
  orders: OrderReader): Promise<UrgentOrder[]>
```

4 · COMPROBAR Y ENTREGAR

```
$ npm test && npm run lint:arch
$ git add -A && git commit -m "feat(orders): …"
$ git push -u origin s2/<tu-nombre>
```

Comportamiento esperado (tests)

- Solo pendientes de más de 3000 € con IVA
- 3000 € sin IVA → total 3630 €
- Ordenados de mayor a menor importe
- Lista vacía si no hay urgentes

Usa el dominio: isUrgent y calculateTotal. No dupliques reglas.

Ampliación

Prompt P1 sobre OrderRow.tsx o docs/arquitectura-<tu-nombre>.md.

ENTREGA Pull Request con la checklist SOLID de la plantilla completada. Sin permisos de escritura: fork y PR desde tu fork.

Solución: reglas de aplicación, infraestructura y UI

5 MIN

Compara con tu eslint.architecture.js · Repositorio de referencia: imagina-clean-react-s2-solucion

eslint.architecture.js · SOLUCIÓN

```
{ files: ['src/orders/application/**/*.{ts,tsx}'],
  rules: {
    'no-restricted-imports': ['error', { patterns: [
      FRAMEWORKS, layer('infrastructure'), layer('ui'),
      composition] }],               // sin layer('application')
    'no-restricted-globals': ['error', ...BROWSER_GLOBALS] } },

{ files: ['src/orders/infrastructure/**/*.{ts,tsx}'],
  rules: { 'no-restricted-imports': ['error', { patterns: [
      layer('ui'), composition] }] } },   // fetch aquí es legítimo

{ files: ['src/orders/ui/**/*.{ts,tsx}'],
  rules: { 'no-restricted-imports': ['error', { patterns: [
      layer('infrastructure'),
      { group: ['axios'], message: 'La presentación no habla HTTP…' }] }] } },
```

RESULTADO

```
$ npm run test:arch
 ✓ architecture.test.ts
   (9 tests)
```

Errores frecuentes

- Patrones con ruta del proyecto (src/orders/…): el patrón se compara con el texto del import
- Dos bloques con la misma regla para la misma carpeta: el último sustituye, no suma

CLAVE Cada capa prohíbe lo que está más afuera o al lado. Ninguna se prohíbe a sí misma.

Solución: listUrgentOrders y su fake

5 MIN

Tres tests en verde, sin red y sin React

SOLUCIÓN

```
// test · paso 1
function createFakeOrderReader(orders: Order[]): OrderReader {
  return { findAll: async () => orders };
}

// list-urgent-orders.ts · paso 3
export async function listUrgentOrders(orders: OrderReader) {
  const all = await orders.findAll();
  return all
    .filter(isUrgent)                  // ya incluye 'pending'
    .map((o) => ({ id: o.id, customerName: o.customerName,
                   total: calculateTotal(o) }))
    .sort((a, b) => b.total - a.total);
}
```

RESULTADO EN LA SOLUCIÓN

```
$ npm test && npm run lint:arch
 ✓ src/orders/application/list-urgent-orders.test.ts (3 tests)
      Tests  44 passed (44)
```

1 No dupliques la regla

isUrgent ya filtra pendientes.

2 Total con IVA

3000 × 1,21 = 3630. El test lo fija.

3 lint:arch en verde

El caso de uso no importa nada prohibido.

DIP El fake es un objeto que cumple el puerto: no hace falta ninguna librería de mocks para probar un caso de uso.

Revisión con la checklist SOLID

5 MIN

Aplicamos la plantilla de Pull Request a una entrega real del grupo

Pregunta	Evidencia en la solución	Comprobación
S · actores	Solo negocio: qué es urgente y en qué orden	Lectura del archivo
O · extender	Cambiar el umbral = cambiar URGENT_FROM en el dominio	Test del dominio
L · contrato	El fake resuelve una lista, como el adapter HTTP	Tests del caso de uso
I · datos	Devuelve 3 campos, no el pedido completo	Tipo UrgentOrder
D · núcleo	Depende de OrderReader, no de fetch	npm run lint:arch
+ abstracción	Ninguna interfaz nueva: el puerto ya existía	git diff --stat

Review útil

- Discute dependencias y coste de cambio
- Cada «sí» lleva evidencia
- Se apoya en comandos, no en gustos

PREGUNTA ¿Qué fila de la checklist te ha costado más justificar con evidencia?

Proyecto final: hito de la Sesión 2

5 MIN

Feature Pedidos de OrderDesk, transformada de forma incremental sesión a sesión

S1	S2	S3	S4	S5
Diagnóstico y matriz	Reglas, checklist y capas	Estructura y reglas puras	Casos de uso, puertos y Composition Root	Hooks, view models, estado y routing

Hecho hoy

- Reglas de importación por capa + tests
- Caso de uso listUrgentOrders con su fake
- Checklist SOLID en la plantilla de Pull Request
- Instrucciones de Copilot por ruta

COMPLETA EL HITO

```
# Tarea antes de la Sesión 3
$ cp docs/arquitectura-objetivo.md \
     docs/arquitectura-<tu-nombre>.md

| Responsabilidad | Evidencia | Capa | 1er paso |
# ≥ 6 filas del código heredado
# Punto de partida: tu matriz de deuda
```

MAÑANA Sesión 3: capas globales, feature modular o Vertical Slice. Tu tabla de clasificación será el mapa.

Test de conceptos: 5 preguntas, 30 segundos

5 MIN

Individual · Responde en el chat al final: 1A, 2C… · 1 punto por acierto

1. Según R. C. Martin, Single Responsibility significa que un módulo…

A) hace una sola cosa B) responde ante un único actor C) tiene menos de 100 líneas

2. ¿Qué consigue STATUS_VIEW con satisfies Record<OrderStatus, StatusView>?

A) el compilador avisa si falta un estado B) mejora el rendimiento C) evita importar React

3. Un adapter lanza un error donde el contrato del puerto promete null. Incumple…

A) Interface Segregation B) Open/Closed C) Liskov

4. En Dependency Inversion, el puerto OrderReader lo define…

A) el adapter HTTP B) el núcleo que lo usa C) el backend

5. ¿En qué capa vive el mapper que traduce ord_st a status?

A) infraestructura B) dominio C) presentación

RANKING Los puntos se suman a las dinámicas de hoy. El ranking se anuncia al inicio de la Sesión 3.

Feedback individual y próxima sesión

5 MIN

Jueves 12 de noviembre · 17:00 – 20:00 · Vertical Slice y modelado de dominio

FEEDBACK · MENSAJE PRIVADO EN EL CHAT

1. La idea que te llevas de hoy

2. El concepto que quieres repasar mañana

Tu Pull Request s2 recibirá un comentario individual.

Tres ideas de hoy

- Acoplamiento = coste de cambio; se mide
- SOLID en React: un actor, tablas exhaustivas, contratos, props mínimas, puertos
- La regla de dependencia se defiende con ESLint y tests

Antes de la Sesión 3

- Completa docs/arquitectura-<tu-nombre>.md
- Sube tu rama s2/<tu-nombre>
- npm test y npm run lint:arch en verde

Sesión 3 · 12 de noviembre

- Repaso
- Arquitectura por features y Vertical Slice
- Modelado de dominio: entidades y patrones

CONTINUIDAD Tu tabla de clasificación es el punto de partida de mañana: decidiremos dónde vive cada pieza.